



Universidade Federal do Pará
Instituto de Ciências Exatas e Naturais
Programa de Pós-Graduação em Ciência da Computação

Relatório de Análise do Copilot para Geração de Código com Acessibilidade
Autor: Luciano A. Teran

Belém, 25 de Dezembro de 2023

1. Geração de Códigos do Projeto

Essa seção apresenta as interações do programador para a geração do código do projeto. Nesse contexto, foi seguido um fluxo real de desenvolvimento. É importante destacar que nessas interações não solicitou-se a geração de códigos com acessibilidade.

1.1 Início

Data de Implementação

10/12/2023

Comando de solicitação

Criar um título, um card no padrão tailwind contendo um título e uma descrição

Cenário

Início

Evidência

Figura 1. Criação da Página de Início

```
src > pages > TS index.tsx > ...
1 | import React from "react";
2 | import Layout from "../components/Layout";
3 |
4 | export default function Home() {
5 |   return (
6 |     <Layout title="TESTE">
7 |       <div>
8 |         <h1 className="text-2xl font-bold mb-4">Title</h1>
9 |         <div className="bg-white shadow-md rounded-lg p-4">
10 |           <h2 className="text-lg font-bold mb-2">Card Title</h2>
11 |           <p className="text-gray-600">Card Description</p>
12 |         </div>
13 |       </div>
14 |     </Layout>
15 |   );
16 | }
17 |
```

Observação

O código foi gerado com um import do React automático. Segui desenvolvendo a página e percebi que não estava conseguindo aplicar recuo. Nesse contexto utilizei o comando “preciso que a lista tenha recuo na lista”. Assim consegui deixar no padrão.

1.2 Considerações de Design e Avaliação

Data de Implementação

10/12/2023

Comando de solicitação

a div deve ter 3 colunas e responsiva no padrão tailwind

Cenário

Considerações de Design e Avaliação

Evidência

Figura 2. Criação da Página de Considerações de Design e Avaliação

```
src > pages > include > TS considerations.tsx > ConsiderationsPage
1  import Layout from "@components/Layout";
2  import { considerationsByCategory } from "../../db/considerationsByCategory";
3  import { dummyCategories } from "../../db/dummyCategories";
4  import ConsiderationCard from "@components/basics/ConsiderationCard";
5
6  export default function ConsiderationsPage() {
7    return (
8      <Layout title="Considerações de Design e Avaliação">
9
10       <div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 gap-4">
11         {considerationsByCategory().map((consideration) => {
12           return (
13             <ConsiderationCard
14               key={consideration.id}
15               id={consideration.id}
16               title={consideration.title}
17               category={consideration.category}
18               content={consideration.content}
19             />
20           )
21         })}
22       </div>
23     </Layout>
24   )
25 }
```

Observação

O componente foi implementado automaticamente, sendo necessário realizar apenas mudanças na inserção da categoria e link para mais informações. Na página foi utilizado o comando “a div deve ter 3 colunas e responsiva no padrão tailwind”. Dessa forma o padrão foi alterado para o solicitado.

1.3 Filtro das Considerações de Design e Avaliação

Data de Implementação

10/12/2023

Comando de solicitação

refatorar código para implementar uma lista que ao clicar em um link filtre a consideration.category

Cenário

Considerações de Design e Avaliação > Filtro

Evidência

Figura 3. Criação do Filtro de Considerações de Design e Avaliação

```
<div className="flex gap-3 mt-5 mb-8">
  {dummyCategories.map((category) => (
    <button
      style={{ backgroundColor: "#005A9F" }}
      className="bg-blue-500 text-white px-2 py-1 rounded-md"
      key={category}
      onClick={() => handleCategoryClick(category)}
    >
      {category}
    </button>
  ))}
  <button
    style={{ backgroundColor: "#005A9F" }}
    className="bg-blue-500 text-white px-2 py-1 rounded-md"
    key={"Todos"}
    onClick={() => setSelectedCategory("")}
  >
    Todos
  </button>
</div>
```

Observação

Solicitei que Github Copilot implementasse o filtro para o category e ele forneceu o código corretamente

1.4 Busca das Considerações de Design e Avaliação

Data de Implementação

10/12/2023

Comando de solicitação

refatorar código para implementar uma campo de busca que filtre parte do texto de consideration.title ou consideration.content

Cenário

Considerações de Design e Avaliação > Busca

Evidência

Figura 4. Criação da Página de Busca de Considerações de Design

```
<Layout title="Considerações de Design e Avaliação">
  <div className="flex gap-3 mt-7 mb-5">
    <div className="relative">
      <input
        type="text"
        placeholder="Buscar"
        className="border border-gray-300 rounded-md px-2 py-1 pr-8"
        value={searchText}
        onChange={handleSearchTextChange}
      />
      <div className="absolute right-2 top-1/2 transform -translate-y-1/2">
        <svg width="18" height="19" viewBox="0 0 18 19" fill="none" xmlns="http://www.w3.org/2000/svg">
          <path d="M14.625 8.0625C14.6016 9.70312 14.1328 11.1445 13.2188 12.3867L17.6836 16.8164C17.8945 17.0508 18 17.
        </svg>
      </div>
    </div>
  </div>
</div>
```

Observação

Solicitei que ele implementasse o filtro para o category e ele forneceu o código corretamente. Para tanto, inseri a imagem de lupa manualmente.

1.5 Busca das Considerações de Design e Avaliação

Data de Implementação

11/12/2023

Comando de solicitação

implementar um card no padrão tailwind contendo título, categoria, descrição, uma lista de imagem e uma lista de referências. A lista deve vir de db/dummyConsiderations.js

Cenário

Considerações de Design e Avaliação

Evidência

Figura 5. Criação do Componente de Card

```
return (  
  <Layout>  
    <div className="bg-white rounded-lg shadow-md p-4">  
      <h1 className="text-2xl font-bold mb-2">{consideration?.title}</h1>  
      <p className="text-gray-500 mb-2">{consideration?.category}</p>  
      <p className="mb-4">{consideration?.description}</p>  
      <div className="mb-4">  
        <h2 className="text-lg font-bold mb-2">Images:</h2>  
        <ul>  
          {consideration?.images.map((image, index) => (  
            <li key={index}>{image}</li>  
          ))}  
        </ul>  
      </div>  
      <div>  
        <h2 className="text-lg font-bold mb-2">References:</h2>  
        <ul>  
          {consideration?.references.map((reference, index) => (  
            <li key={index}>{reference}</li>  
          ))}  
        </ul>  
      </div>  
    </div>  
  </Layout>  

```

Observação

Solicitei que ele implementasse o filtro para o category e ele forneceu o código corretamente. Para esse caso, foram realizados alguns ajustes de adequação a forma de apresentação e consumo dos dados do BD.

1.6 Implementação da imagem da página de consideração

Data de Implementação

11/12/2023

Comando de solicitação

criar um componente para imagem

Cenário

Considerações de Design e Avaliação > Consideração

Evidência

Figura 6. Criação do Componente de Card para Considerações

```
<div className="mb-4">
  <ul>
    {consideration.images.map((image, index) => (
      <li key={index}>
        //criar um componente para imagem
        <img src={image.url} alt={image.alt} />
      </li>
    ))}
  </ul>
</div>
</div>
```

Observação

Ao solicitar que fosse gerado um componente de imagem, o Copilot apresentou automaticamente o parâmetro de legenda (alt).

1.7 Criação do componente do card de persona

Data de Implementação

12/12/2023

Comando de solicitação

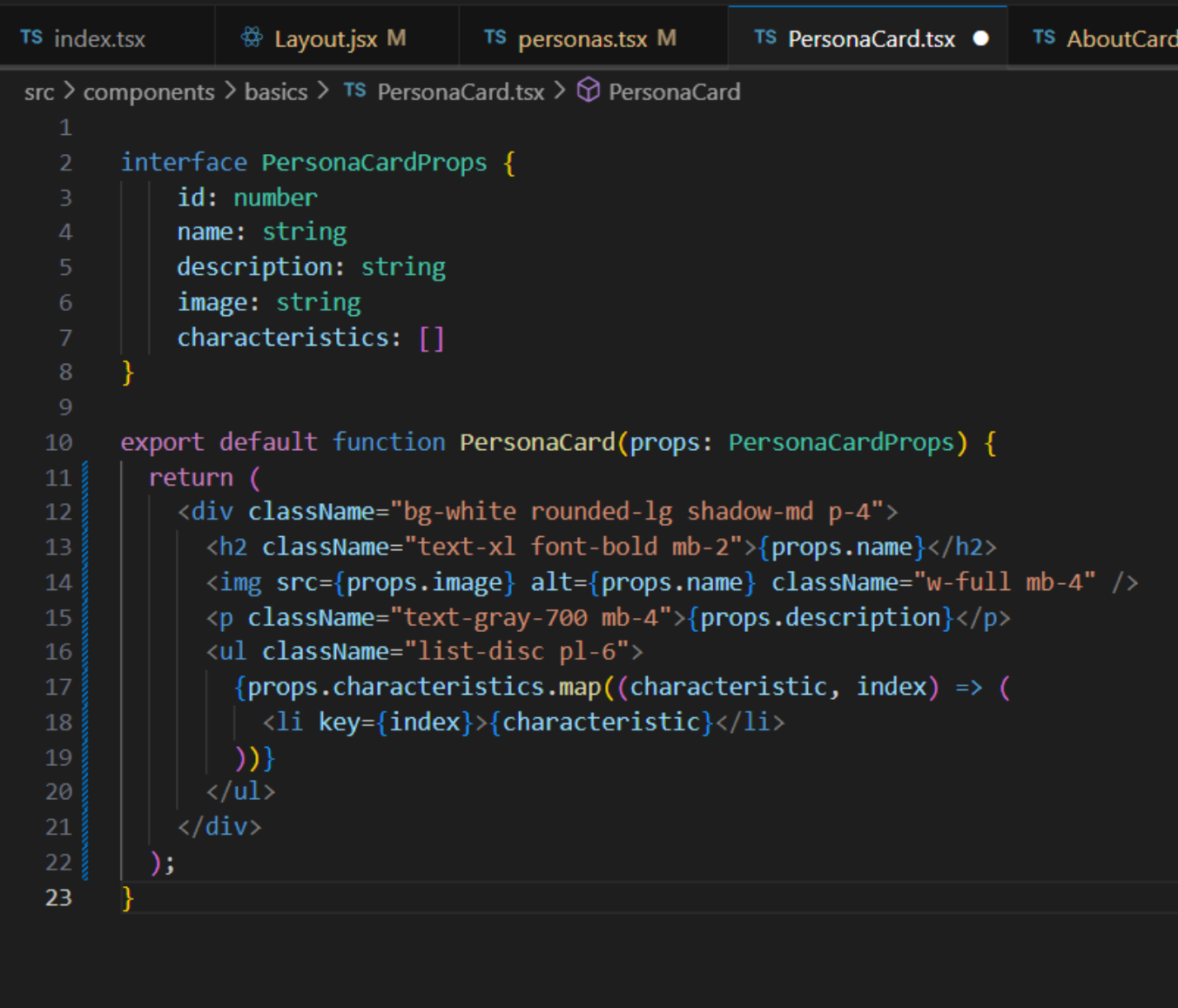
criar um card com tailwind com título, imagem, descrição e uma lista de informações

Cenário

Personas > Componentes

Evidência

Figura 7. Criação do Componente de Card para Persona



```
TS index.tsx  Layout.jsx M  TS personas.tsx M  TS PersonaCard.tsx ●  TS AboutCard

src > components > basics > TS PersonaCard.tsx > PersonaCard

1
2  interface PersonaCardProps {
3      id: number
4      name: string
5      description: string
6      image: string
7      characteristics: []
8  }
9
10 export default function PersonaCard(props: PersonaCardProps) {
11     return (
12         <div className="bg-white rounded-lg shadow-md p-4">
13             <h2 className="text-xl font-bold mb-2">{props.name}</h2>
14             <img src={props.image} alt={props.name} className="w-full mb-4" />
15             <p className="text-gray-700 mb-4">{props.description}</p>
16             <ul className="list-disc pl-6">
17                 {props.characteristics.map((characteristic, index) => (
18                     <li key={index}>{characteristic}</li>
19                 ))}
20             </ul>
21         </div>
22     );
23 }
```

Observação

Foi solicitado um código para construção de um componente de persona, gerado com problemas. Em seguida foi realizado alguns ajustes do componente.

1.8 Criação do componente da página de personas

Data de Implementação

12/12/2023

Comando de solicitação

refatorar para ter duas colunas em desktop e uma coluna para mobile, usando talwind

Cenário

Personas

Evidência

Figura 8. Criação da Página de Persona

```
src > pages > include > TS personas.tsx > PersonasPage
1  import Layout from "@components/Layout";
2  import PersonaCard from "@components/basics/PersonaCard";
3  import { dummyPersonas } from "../../db/dummyPersonas";
4
5  export default function PersonasPage() {
6    return (
7      <Layout title="Consideração de Design e Avaliação">
8        <div className="grid grid-cols-1 md:grid-cols-2 gap-8 mt-7 mb-5">
9          {dummyPersonas.map((personas) => (
10            <PersonaCard
11              key={personas.id}
12              id={personas.id}
13              name={personas.name}
14              image={personas.image}
15              description={personas.description}
16              characteristics={personas.characteristics}
17            />
18          ))}
19        </div>
20      </Layout>
21    )
22  }
```

Observação

Foi solicitado um código para construção de tela responsiva, tendo duas colunas para telas maiores e uma coluna para telas menores. Em seguida implementou-se o componente de card de personas, sem considerar o suporte do Github Copilot.

1.9 Criação do componente do card de sobre

Data de Implementação

12/12/2023

Comando de solicitação

criar um componente que tenha uma imagem redonda, um título, com duas descrições separadas

Cenário

Sobre

Evidência

Figura 9. Criação do Componente de Card para Sobre

```
src > components > basics > TS AboutCard.tsx > AboutCard
1 | import React from 'react';
2 |
3 | interface AboutCardProps {
4 |   id: number;
5 |   name: string;
6 |   description1: string;
7 |   description2: string;
8 |   email: string;
9 |   image: string;
10 | }
11 |
12 | export default function AboutCard(props: AboutCardProps) {
13 |   return (
14 |     <div>
15 |       <img src={props.image} alt={props.name} style={{ borderRadius: '50%' }} />
16 |       <h2>{props.name}</h2>
17 |       <p>{props.description1}</p>
18 |       <p>{props.description2}</p>
19 |     </div>
20 |   );
21 | }
```

Observação

Foi observado que o primeiro comando gerou um código com erros de sintaxe. Com isso, foi solicitado novos códigos para a correção dos erros.

1.10 Criação do componente da página de sobre

Data de Implementação

13/12/2023

Comando de solicitação

Criar um card no padrão do talwind contendo na seguinte ordem: um subtítulo, um componente AboutCard, um subtítulo, um subtítulo, um subtítulo menor, dois textos descritivos e um mapa da google estático

Cenário

Sobre

Evidência

Figura 10. Criação do Página de Sobre

```
1
2 import Layout from "@components/Layout";
3 import AboutCard from "@components/basics/AboutCard";
4
5 export default function AboutPage() {
6   return (
7     <Layout title="Sobre">
8       <div className="bg-white rounded-lg shadow-lg p-6 mt-7">
9         <h2 className="text-2xl font-bold mb-4">Subtitulo</h2>
10        <AboutCard
11          id={1}
12          name="Titulo"
13          description1="Texto descritivo 1"
14          description2="Texto descritivo 2"
15          email="email@teste.com"
16          image="https://www.w3schools.com/howto/img_avatar.png"
17        />
18        <h2 className="text-2xl font-bold mb-4">Subtitulo</h2>
19        <h3 className="text-lg font-bold mb-2">Subtitulo menor</h3>
20        <p className="mb-2">Texto descritivo 1</p>
21        <p className="mb-4">Texto descritivo 2</p>
22        
    <div className="flex flex-col">
      <h2 className="text-lg font-bold mb-1">Configure as Perguntas</h2>
      <span className="mb-5 italic">Ajuste as próximas seções desse recurso</span>
      <h3 className="text-md font-bold mb-1">Quantidade de Perguntas</h3>
      <span className="mb-3 italic">Máximo {questionsMax} Perguntas</span>
      <label className="mb-5">
        <input className="border border-gray-300 rounded-md px-2 py-1"
          type="number"
          onChange={(e) => {
            setSetting({ ...setting, quantity: parseInt(e.target.value) });
          }}
          min={1}
          max={questionsMax} />
      </label>
      <h3 className="text-md font-bold mb-1">Cenários</h3>
      <div className="flex flex-col">
        <label className="mb-2">
          <input className="mr-2" type="checkbox" name="category" checked={setting.category} />
          Category 1
        </label>
        <label>
          <input className="mr-2" type="checkbox" name="category" checked={setting.category} />
          Category 2
        </label>
      </div>
    </div>
  </div>
);
```

Figura 12. Criação da Estrutura de Perguntas

```
{step > setting.quantity && (
  <div className="flex flex-col">
    <div className="flex flex-col">
      <h2 className="text-lg font-bold mb-1">Perguntas Finalizadas</h2>
    </div>
    <div className="flex flex-col">
      <span className="mb-3 italic">Lorem ipsum dolor sit am</span>
    </div>
    <div>
      <button style={{backgroundColor:"#005A9F"}} onClick={() => setStep(0)} className="bg-blue-500 hover:bg-blue-700 text-white py-2 px-4">
        Refazer
      </button>
    </div>
  </div>
);
</div>
</Layout>
```

Observação

Foi solicitado o suporte para a geração de uma tela de perguntas e respostas, na qual a demanda da estrutura foi atendida parcialmente devido a complexidade da tela. Nesse primeiro momento utilizou-se o modo exploratório para gerar a seção de configuração. Contudo, a partir de então utilizou-se o modo acelerado, e nesse processo apareceram algumas sugestões para as funções de controle das questões geradas em modo aleatório e para construir a estrutura do html. Nesse contexto as duas foram aceitas.

1.12 Conclusões

Minhas principais experiências foram voltadas para a acessibilidade e o desenvolvimento front-end de sistemas. O Copilot tem se tornado uma ferramenta que é utilizada constantemente em minhas atividades de desenvolvimento.

Na tarefa de início utilizei o modo misto, quando solicitei a geração de código a partir do comando “Criar um título, um card no padrão tailwind contendo um título e uma descrição”. A solicitação não foi totalmente atendida, então precisei refatorar o código até chegar próximo do que eu queria.

Para a tarefa de considerações de design e avaliação utilizei o modo misto, dentre os comandos, inicialmente, solicitei que o Copilot por meio do comando “a div deve ter 3 colunas e responsiva no padrão tailwind”. Em seguida, utilizei o modo acelerado e o exploratório para gerar o filtro de cartões da plataforma. Percebi que o Copilot não gerou recursos como o aria, para avisar alterações em tela ou também. Todavia, ele gerou o alt para a imagem da tela de categoria.

Nessa tarefa também utilizei o modo misto. A sugestão do Copilot atendeu quase por completo a geração do recurso de Personas. Assim, realizei alguns ajustes solicitando que o card fosse gerado no padrão do Tailwind.

Desenvolvi o recurso de perguntas e respostas da plataforma pelo modo misto, inicialmente pelo modo exploratório. Em seguida, realizei os ajustes necessários na estrutura da página e aceitei algumas sugestões do copilot no modo corrido. Voltei a utilizar o modo sugestão na geração das funções da tela que, em seguida, deram erros de implementação. Dessa forma, realizei os ajustes necessários de forma manual para implementar a função. Nesse processo o Copilot sugeriu alguns códigos, que foram aceitos somente quando eram de completude do código.

O último recurso desenvolvido foi o Sobre, feito por meio do modo misto. Contudo, a estrutura segura estava correta, realizado apenas ajustes de estilo com as classes no padrão tailwind. Por não termos uma chave de API para o Google Maps, utilizamos o modo incorporação de iframe da ferramenta.

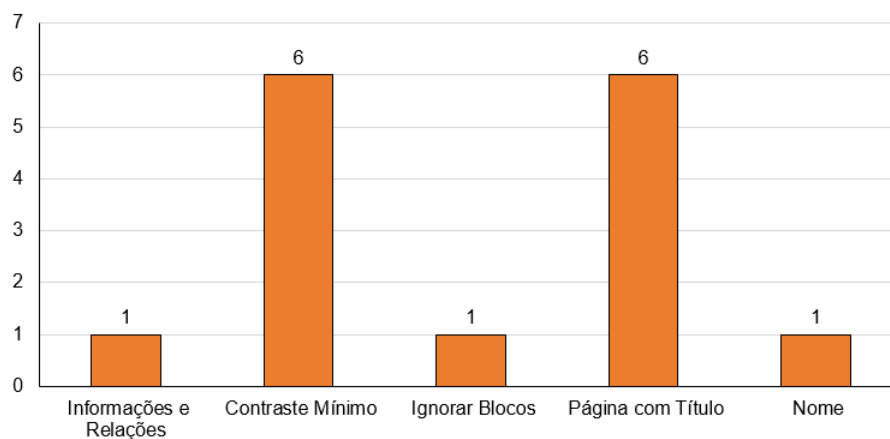
Durante a construção dessas telas, percebi que algumas sugestões do Copilot não me atendiam ou geravam bugs. Nesse processo, refiz algumas funções de comportamentos da tela ou implementei classes do Tailwind para refatorar código. Além disso, antes da refatoração também solicitei outras sugestões do Github Copilot. Essas sugestões não me atenderam e, desse modo, foi necessário refatorar mais manualmente.

Observei que o Github Copilot gerou texto alternativo sempre que uma sugestão de código incorporava uma imagem em sua estrutura. Porém, o Github Copilot não gerou outros recursos necessários, tais como o nome dos inputs ou atributos do Accessible Rich Internet Applications (Aria).

Percebi que o Github Copilot não forneceu tantas sugestões para a acessibilidade. Ou seja, ele apresentou pequenas ou nenhuma sugestões. Além disso, a falta de avisos dos recursos implementados por afetar a experiência de programadores iniciantes, seja no desenvolvimento de código de modo geral ou códigos voltados para o front-end e sistemas acessíveis. Portanto, isso pode afetar o desenvolvimento sobre a acessibilidade e, consequentemente, a conscientização da temática.

Ao observar os testes automatizados (Figura 13), percebi que a principal violação foi pela falta de Título Inicial nas páginas desenvolvidas e Contraste Mínimo do menu, o que afetou negativamente os critérios de sucesso 2.4.2 e 1.4.3, respectivamente. Desse modo, foram coletadas 6 violações relacionadas para esses aspectos. Além disso, os dados destacam uma violação para o critério Informação e Relações, critério de sucesso 1.3.1; uma violação para Ignorar Blocos, critério de sucesso 2.4.1; e uma violação para Nome, critério 4.1.2.

Figura 13. Violações dos Critérios de Sucesso da WCAG



É importante indicar que essas violações foram mapeadas do nível AA, que é o recomendado pela WCAG para atender diversos usuários no acesso às plataformas web.

2. Refatoração para a Acessibilidade

Essa seção apresenta as interações do programador para a geração do código do projeto com acessibilidade. Desse modo, os códigos foram refatorados a partir de solicitações de geração de código acessível.

2.1 Resolução da Violação de Título

Data de Implementação

16/12/2023

Comando de solicitação

Refatorar o código para estar adequada a norma de acessibilidade da wcag

Cenário

Início, Considerações de Design, Personas, Perguntas e Resposta e Sobre

Evidência

Figura 14. Primeira Tentativa de Refatoração do Título das Páginas

```
1  import React from "react";
2  import Sidebar from "../Sidebar";
3
4  const Layout = ({ children, title }) => {
5    return (
6      <div className="h-screen flex flex-row justify-start">
7        <Sidebar />
8        <div
9          style={{
10             backgroundColor: "#D9D9D9",
11             height: "fit-content",
12             minHeight: "100%",
13           }}
14          className="flex-1 p-7"
15          role="main"
16          aria-label={title}
17        >
18          <div className="text-white">
19            <h1 className="text-2xl font-semibold" style={{ color: "#284863" }}>
20              {title}
21            </h1>
22          </div>
23          <div>{children}</div>
24        </div>
25      </div>
26    );
27  };
28
29  export default Layout;
```

O comando acima não forçou uma sugestão adequada para o problema de título. Contudo, o código não solucionou o problema de acessibilidade.

Data de Implementação

16/12/2023

Comando de solicitação

Refatorar o código para estar a norma de acessibilidade da wcag: A title should be provided for the document, using a non-empty title element in the head section.

Cenário

Início, Considerações de Design, Personas, Perguntas e Resposta e Sobre

Evidência

Figura 15. Segunda Tentativa de Refatoração do Título das Páginas

```
1 import React from "react";
2 import Sidebar from "../Sidebar";
3
4 const Layout = ({ children, title }) => {
5   return (
6     <div className="h-screen flex flex-row justify-start">
7       <Sidebar />
8       <div
9         style={{
10           backgroundColor: "#D9D9D9",
11           height: "fit-content",
12           minHeight: "100%",
13         }}
14         className="flex-1 p-7"
15         role="main"
16         aria-label={title}
17       >
18         <div className="text-white">
19           <h1 className="text-2xl font-semibold" style={{ color: "#284863" }}>
20             {title}
21           </h1>
22         </div>
23         <div>{children}</div>
24       </div>
25     </div>
26   );
27 }
28
29 export default Layout;
```

Data de Implementação

16/12/2023

Comando de solicitação

Refatorar o código para estar a norma da acessibilidade wcag para atender o critério de sucesso 2.4.2 em react

Cenário

Início, Considerações de Design, Personas, Perguntas e Resposta e Sobre

Evidência

Figura 16. Terceira Tentativa de Refatoração do Título das Páginas

```
src > components > @ Layout.jsx > @ default
1 import React from "react";
2 import Sidebar from "../Sidebar";
3
4 import React from "react";
5 import Sidebar from "../Sidebar";
6
7 const Layout = ({ children, title }) => {
8   return (
9     <div className="h-screen flex flex-row justify-start">
10       <Sidebar />
11       <div
12         style={{
13           backgroundColor: "#D9D9D9",
14           height: "fit-content",
15           minHeight: "100%",
16         }}
17         className="flex-1 p-7"
18       >
19         <div className="text-white">
20           <h1 className="text-2xl font-semibold" style={{ color: "#284863" }}>
21             {title}
22           </h1>
23         </div>
24         <div>{children}</div>
25       </div>
26     </div>
27   );
28 }
29
30 export default Layout;
```

O comando acima também não gerou um resultado esperado. Logo, as violações de acessibilidade persistiram após a inspeção.

Data de Implementação

16/12/2023

Comando de solicitação

Refatorar o código para corrigir o problema de acessibilidade "A title should be provided for the document, using a non-empty title element in the head section."

Cenário

Início, Considerações de Design, Personas, Perguntas e Resposta e Sobre

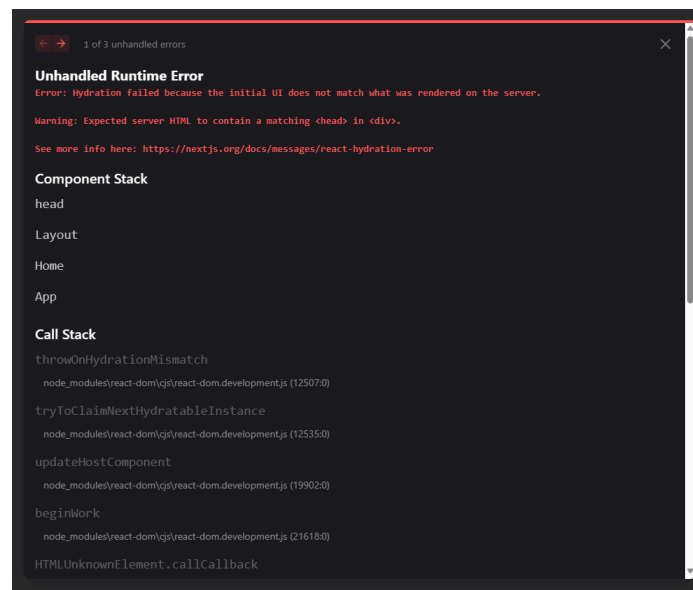
Evidência

Figura 17. Quarta Tentativa de Refatoração do Título das Páginas

```
src > components > Layoutjsx > Layout
1 import React from "react";
2 import Sidebar from "../Sidebar";
3
4 const Layout = ({ children, title }) => {
5   return (
6     <>
7       <head>
8         <title>{title}</title>
9       </head>
10      <div className="h-screen flex flex-row justify-start">
11        <Sidebar />
12        <div style={{backgroundColor: "#D9D9D9", height: "fit-content", minHeight: "100%"}} className=
13          <div className="text-white">
14            <h1 className="text-2xl font-semibold" style={{color: "#284863"}}>{title}</h1>
15          </div>
16          <div>
17            {children}
18          </div>
19        </div>
20      </div>
21    </>
22  );
23 };
24
25 export default Layout;
```

A violação de acessibilidade foi resolvida após solicitar ao Copilot a correção de acessibilidade para um problema passado entre aspas. Logo, a sintaxe favoreceu para resolver esse problema no título. Além disso, foi necessário indicar onde a alteração deveria ser aplicada. Contudo, um novo problema foi gerado e verificado que a estrutura do framework não aceitava esse comando específico.

Figura 18. Quarta Tentativa de Refatoração do Título das Páginas



Assim, o problema de sintaxe do framework React foi gerado por meio de da inserção do componente Head.

Figura 19. Quinta Tentativa de Refatoração do Título das Páginas

```
src > components > Layout.jsx > Layout
1  import React from "react";
2  import Sidebar from "../Sidebar";
3  import Head from "next/head";
4
5  const Layout = ({ children, title }) => {
6    return (
7      <>
8        <Head>
9          <title>{title}</title>
10        </Head>
11        <div className="h-screen flex flex-row justify-start">
12          <Sidebar />
13          <div style={{backgroundColor: "#D9D9D9", height: "fit-content", minHeight: "100%}} className=
14            <div className="text-white">
15              <h1 className="text-2xl font-semibold" style={{color: "#284863"}}>{title}</h1>
16            </div>
17            <div>
18              {children}
19            </div>
20          </div>
21        </div>
22      </>
23    );
24  };
25
26
27  export default Layout;
```

Data de Implementação

16/12/2023

2.3 Resolução das Violações em Campos de Texto

Comando de solicitação

Refatorar o código para corrigir o problema de acessibilidade "This textinput element does not have a name available to an accessibility API. Valid names are: label element, title undefined, aria-label undefined, aria-labelledby undefined."

Cenário

Considerações de Design

Evidência

Figura 20. Primeira Tentativa de Refatoração em Campos de Texto

```
20
21
22   const filteredConsiderations = considerationsByCategory().filter(
23     (consideration) =>
24       (selectedCategory === "" || consideration.category === selectedCategory) &&
25       (searchText === "" ||
26         consideration.title.toLowerCase().includes(searchText.toLowerCase()) ||
27         consideration.content.toLowerCase().includes(searchText.toLowerCase()))
28   );
29
30   return (
31     <Layout title="Considerações de Design e Avaliação">
32       <div className="flex gap-3 mt-7 mb-5">
33         <div className="relative">
34           <input
35             type="text"
36             placeholder="Buscar"
37             className="border border-gray-300 rounded-md px-2 py-1 pr-8"
38             value={searchText}
39             onChange={handleSearchTextChange}
40             name="searchText" // Add name attribute for accessibility
41             aria-label="Buscar" // Add aria-label attribute for accessibility
42           />
43           <div className="absolute right-2 top-1/2 transform -translate-y-1/2">
44             <svg width="18" height="19" viewBox="0 0 18 19" fill="none" xmlns="http://www.w3.org/2000/svg">
45               <path d="M14.625 8.0625C14.6016 9.70312 14.1328 11.1445 13.2188 12.3867L17.6836 1"
46             />
47           </div>
48         </div>
49       <div className="flex gap-3 mt-5 mb-8">
50         {dummyCategories.map((category) => (
```

Na página de considerações de design e avaliação o código gerado apresentou duas violações, sendo elas: “WCAG2AA.Principle4.Guideline4_1.4_1_2.H91.InputText.Name” e “WCAG2AA.Principle1.Guideline1_3.1_3_1.F68”. Essas violações estavam relacionadas aos nome, informações e relações de campos de entrada de dados. Em contrapartida, ao resolver o primeiro, o GitHub Copilot resolveu automaticamente o segundo. Além disso, ele apresentou sugestões de campo.

2.4 Resolução das Violações em Iframes

Data de Implementação

16/12/2023

Comando de solicitação

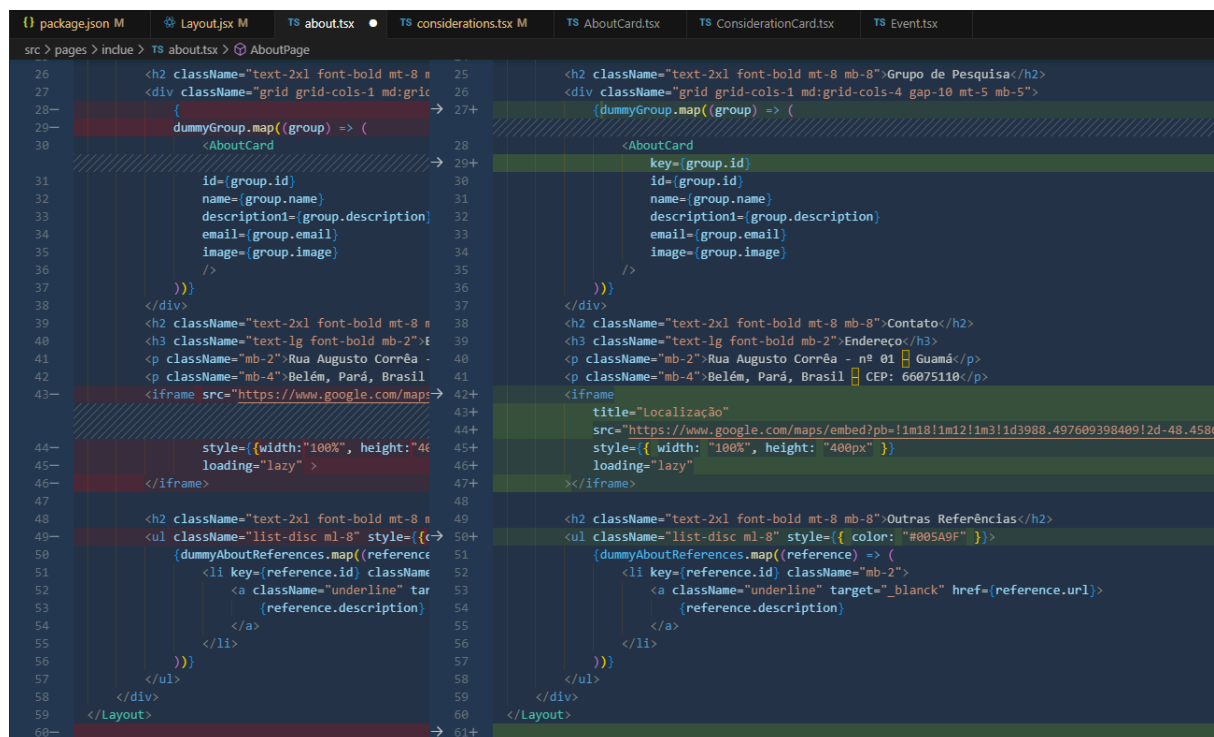
Refatorar o código para corrigir o problema de acessibilidade "Iframe element requires a non-empty title attribute that identifies the frame."

Cenário

Sobre

Evidência

Figura 21. Primeira Tentativa de Refatoração no Iframe



A sugestão de código resolveu a problemática do problema da página de sobre. O problema foi solucionado por meio da sugestão de passar uma solicitação e descrição do problema fornecido pelo pal1y.

2.5 Resolução das Violação de Contraste

Data de Implementação

17/12/2023

Comando de solicitação

Refatorar o código para corrigir o problema de acessibilidade "This element has insufficient contrast at this conformance level. Expected a contrast ratio of at least 4.5:1, but text in this element has a contrast ratio of 4.38:1. Recommendation: change background to #f7f8fa."

Cenário

Considerações de Design

Evidência

Figura 22. Primeira Tentativa de Refatoração do Contraste

```
50     return (  
51         <layout title="Perguntas e Respostas">  
52             <step> 0 && step <- setting.quantity && (  
53                 <div className="underline flex items-center gap-1 mt-4" onClick={() => setStep(step-1)}>  
54                     <svg width="18" height="15" viewBox="0 0 18 15" fill="none" xmlns="http://www.w3.org/2000/svg">  
55                         <path d="M0.351562 6.60156L6.60156 0.351562C6.86198 0.117188 7.16146 0 7.5 0C7.83854 0 8.13802 0.117188 8.39844 0.351562C8.63281 0.611979 8.75 0.911458 8.75 1.25C8.75 1.58854 8.63281 1.88802 8.39844 2.14  
56                         </path>  
57                     </svg>  
58                     Voltar  
59                 </div>  
60                 <div className="bg-white shadow-md rounded-lg p-4 mt-7 mb-5"> → 60+ <div className="bg-white shadow-md rounded-lg p-4 mt-7 mb-5" style={{ backgroundColor: "#f7f8fa" }}>  
61                     <div className="flex flex-col">  
62                         <div className="flex flex-col">  
63                             <div className="flex flex-col">  
64                                 <h2 className="text-lg font-bold mb-1">Configure as Perguntas</h2>  
65                                 <span className="mb-5 italic">Ajuste as próximas seções desse recurso</span>  
66                                 <h3 className="text-md font-bold mb-1">Quantidade de Perguntas</h3>  
67                                 <span className="mb-3 italic">Máximo {questionsMax} Perguntas</span>  
68                                 <input type="text" value={setting.quantity} className="border border-gray-300 rounded-md px-2 py-1">  
69                                 <input type="number" value={setting.quantity} className="border border-gray-300 rounded-md px-2 py-1">  
70                                 <input type="text" value={setting.quantity} className="border border-gray-300 rounded-md px-2 py-1">  
71                                 <input type="number" value={setting.quantity} className="border border-gray-300 rounded-md px-2 py-1" onChange={(e) => {  
72                                     setting.quantity = parseInt(e.target.value);  
73                                     setSetting({ ...setting, quantity: parseInt(e.target.value) });  
74                                     min=1  
75                                     max={questionsMax} />  
76                                 </div>  
77                             </div>  
78                         </div>  
79                     </div>  
80                 </div>  
81             </step>  
82         </layout>  
83     );
```

O código informado não gerou a sugestão esperada. Logo, a violação de contraste persistiu.

Data de Implementação

17/12/2023

Comando de solicitação

Refatorar o código para corrigir o problema de acessibilidade "Refatorar o código para corrigir o problema de acessibilidade "This element has insufficient contrast at this conformance level. Expected a contrast ratio of at least 4.5:1, but text in this element has a contrast ratio of 4.38:1." localizado em "Perguntas e Respostas" seguindo a recomendação "change background to #f7f8fa !important."

Cenário

Considerações de Design

Evidência

Figura 23. Segunda Tentativa de Refatoração do Contraste

```
98
99
100 <div className="flex flex-col items-start mt-24">
101   {menuItems.map(({ icon: Icon, ...menu }) => {
102     const classes = getNavItemClasses(menu);
103     return (
104       <div className={classes}>
105         <Link href={{ pathname: menu.link }}>
106           <div className="flex py-4 px-3 items-center w-full h-full">
107             <div style={{ width: "2.5rem" }}>
108               <Icon />
109             </div>
110             <!toggleCollapse && (
111               <span
112                 className={classNames("text-md font-medium text-text-light", {
113                   "bg-light-lighter": activeMenu.id === menu.id,
114                 })}
115                 style={{ backgroundColor: activeMenu.id === menu.id ? "#f7f8fa" : "" }}
116               >
117                 {menu.label}
118               </span>
119             </div>
120           </Link>
121         </div>

```

O código informado gerou a sugestão esperada. Porém, o conteúdo não foi adequado, ao gerar mais de uma camada de cor no background. A propriedade “!important” não é recomendada, pois afeta o comportamento do projeto na escalabilidade dos estilos e interações construídas por frameworks visuais, tais como o tailwind e bootstrap, ou aplicação de comandos css.

Data de Implementação

17/12/2023

Comando de solicitação

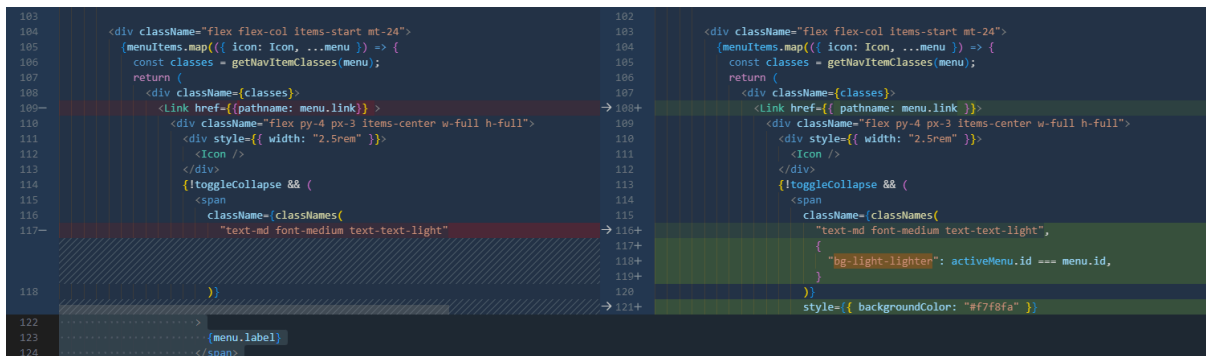
Refatorar o código para corrigir o problema de acessibilidade “This element has insufficient contrast at this conformance level. Expected a contrast ratio of at least 4.5:1, but text in this element has a contrast ratio of 4.38:1.” localizado em “Perguntas e Respostas” seguindo a recomendação “change background to #f7f8fa and remove other property if necessary.”

Cenário

Considerações de Design

Evidência

Figura 24. Terceira Tentativa de Refatoração do Contraste



Por fim, para deixar o comando mais genérico, foi aumentado o comando de entrada solicitando que o copilot removesse qualquer propriedade ou classe que não fosse adequada. Mas não foi suficiente e o problema persistiu. Portanto, o ajuste foi feito manualmente ao remover a propriedade “text-text-light” e adicionando a propriedade “style={{ color: “#284B63” }}” na tag span.

2.6 Conclusão

Testei vários comandos durante a refatoração do projeto, com o objetivo de melhorar a acessibilidade dos recursos desenvolvidos. Observei que alguns problemas foram solucionados de forma totalmente autônoma pelo GitHub Copilot, solucionada de forma parcialmente autônoma ou não foram solucionadas com o suporte do assistente. As observações são detalhadas a seguir:

- Violações de acessibilidade solucionadas pelo GitHub Copilot: Os problemas solucionados de forma autônoma estavam relacionados a incursão de propriedades faltantes ou critérios de sucessos técnicos para atender a acessibilidade. Para explicar esses casos, pode-se citar a inserção do título do iframe da página de sobre e a inserção de nome ou aria-label para campos de entradas de dados dos formulários.
- Violações solucionadas parcialmente pelo GitHub Copilot: Os problemas atendidos parcialmente ocorriam quando eu solicitava ao copilot para corrigir uma violação de acessibilidade. Nesse contexto, o copilot não fornecia uma sugestão que corrigia a acessibilidade do projeto. Como exemplo, pode-se citar o caso ocorrido no título, no qual foi necessário marcar com o mouse a estrutura onde o código deveria ser refatorado.
- Violações não atendidas: Os problemas de acessibilidade não resolvidos devido a necessidade de conceitos de design, arquitetura e acessibilidade de sistemas. Por

exemplo, o erro de contraste não foi resolvido devido o problema ser ocasionado por uma propriedade que deixou o texto do menu na cor cinza. Com isso, foram realizados testes de comandos que em sua estrutura haviam recomendações de cores ou localização do problema, mas que o Copilot não conseguiu interpretar e aplicar um ajuste adequado. Desse modo, realizei o ajuste manual ao remover a classe que causava essa problemática de contraste.

Sobre os comandos utilizados, percebi que o comando mais adequado foi quando eu seguia a seguinte estrutura de código “Refatorar o código para corrigir o problema de acessibilidade “<descrição do problema informado pelo pa11y>”. Esse comando foi aderente para que o Copilot fornecesse sugestões para a resolver a violação de acessibilidade.

Houveram alguns parâmetros necessários para que a sugestão fosse fornecida de forma adequada. Desse modo, o comando precisava da descrição da violação fornecida durante a inspeção de Pa11y. Em segundo lugar, para alguns casos, o Github Copilot não fornecia uma sugestão adequada devido não conhecer onde a alteração deveria ser aplicada. Para isso, realizei a seleção do local onde o ajuste era necessário e, assim, o assistente forneceu uma sugestão adequada.

Percebi que o Github Copilot não fornece sugestões de forma autônoma e, nesse contexto, o assistente inteligente não atende a geração de código acessível. Portanto, foi muito importante eu conhecer como a acessibilidade deve ser aplicada, os frameworks utilizados e as regras de arquitetura do projeto.

3. Contrastes dos Testes de Geração de Código

Os resultados obtidos por meio dos dois testes fizeram eu refletir que o Github não gera código acessível de uma forma abrangente. Os dados do primeiro teste demonstram que o Github Copilot gera problemas de acessibilidade, haja vista que não segue as recomendações da WCAG. Entretanto, o texto alternativo foi fornecido como uma sugestão do assistente inteligente, sendo um recurso essencial para a acessibilidade.

Alguns comentários poderiam ser fornecidos pelo assistente para estimular o conhecimento de acessibilidade do programador. Nesse contexto, por exemplo, um programador pode apagar um código ou propriedade, como o alt, caso não saiba da importância desse recurso. Além disso, o assistente inteligente carece de sugestões que

possam indicar problemas conceituais de acessibilidade. Por exemplo, ao não fornecer um comentário que há problemas de contrastes.

Um fator interessante é indicar no comando de entrada o problema que está relacionado a acessibilidade, indicando o que está gerando a barreira de acesso. Além disso, as vezes é necessário ser mais específico e indicar onde o problema está ocorrendo, para que o Github Copilot forneça uma sugestão adequada para o problema.

Também é necessário ter um conhecimento sólido sobre a arquitetura do projeto e frameworks utilizados, pois as vezes a sugestão não é suficiente ou o assistente inteligente fornece erros no projeto em desenvolvimento.